

Katholieke
Universiteit
Leuven

**Department of
Computer Science**

GEAVANCEERDE COMPUTERARCHITECTUUR

Lab 1

Thibaut Beck
Wannes Paesschesoone

Academic year 2025–2026

Contents

1 Bespreking **1**
1.1 Bespreking van de workflow 1
1.2 Observaties en resultaten 1
1.3 Invloed van het aantal blokken en threads 1
1.4 Questions 1
1.4.1 How would you define the block- and grid size? (# threads in block and # blocks in grid)? 1
1.4.2 How does a thread know which element it should process? 2
1.4.3 What is the largest possible block size (max. threads per block)? 2
1.5 Conclusie 2

2 Code **3**

1 Bespreking

Dit verslag bespreekt de basis van CUDA-programming in C++. Het doel van dit verslag is om aan te tonen hoe een array efficiënt kan worden omgedraaid met behulp van een GPU. Daarbij wordt onderzocht hoe blokken en threads samenwerken om de rekenkracht van de GPU optimaal te benutten.

1.1 Bespreking van de workflow

In het programma worden eerst de nodige variabelen geïnitialiseerd. Vervolgens wordt er een pointer aangemaakt die zal verwijzen naar een geheugenblok op de GPU. Het reserveren van dit geheugen en het toewijzen van de pointer gebeurt met behulp van `cudaMalloc`. Daarna worden de gegevens van de host (de CPU) naar het device (de GPU) gekopieerd via `cudaMemcpy`.
Vervolgens wordt de CUDA-kernel uitgevoerd, waarbij elke thread een eigen index berekent en een deel van de taak uitvoert. Een CUDA-kernel wordt gedeclareerd met het sleutelwoord `__global__`. Wanneer de kernel klaar is met zijn berekeningen, worden de resultaten opnieuw van het device naar de host overgezet.
Tot slot wordt het geheugen dat op het device werd gereserveerd, vrijgegeven met `cudaFree`. Deze stappen vormen samen de standaard workflow van een CUDA-programma, waarbij data efficiënt tussen CPU en GPU worden uitgewisseld. Een goed beheer van deze geheugenoverdrachten is essentieel voor de prestaties, aangezien de communicatie tussen host en device vaak de meeste tijd in beslag neemt.

1.2 Observaties en resultaten

Tijdens het experiment werd vastgesteld dat de array correct wordt omgedraaid op de GPU. De uitvoering gebeurt parallel, waarbij elke thread één of meerdere elementen verwerkt. De prestaties hangen sterk af van de gekozen configuratie van blokken en threads: bij te weinig threads moeten er meerdere kernel-aanroepen gebeuren, wat extra tijd kost, terwijl te veel threads geen merkbare snelheidswinst opleveren. Deze observaties komen overeen met de verwachtingen uit de theorie van GPU-programmering, waar een evenwichtige verdeling van het werk cruciaal is voor efficiënt gebruik van de beschikbare rekenkernen.

1.3 Invloed van het aantal blokken en threads

De GPU verdeelt het werk over meerdere blokken, waarbij elk blok een aantal threads bevat die parallel worden uitgevoerd. Door de berekening van de index (`idx = threadIdx.x + blockIdx.x * blockDim.x`) krijgt elke thread een uniek element van de array toegewezen. Het is belangrijk dat gecontroleerd wordt of de index binnen de grenzen van de array valt, om fouten of onverwacht gedrag te vermijden.

1.4 Questions

1.4.1 How would you define the block- and grid size? (# threads in block and # blocks in grid)?

De blok- en gridgrootte bepalen hoe het werk verdeeld wordt over de GPU. Een block bestaat uit een aantal threads die samenwerken via gedeeld geheugen, terwijl een grid uit meerdere blocks bestaat die onafhankelijk werken.
Het aantal threads per block (`blockDim.x`) en het aantal blocks per grid (`gridDim.x`) wordt meestal gekozen

op basis van de grootte van de dataset en de hardwarekarakteristieken van de GPU. In de praktijk kiest men voldoende threads per block om de GPU volledig te benutten (bijvoorbeeld 128, 256 of 512 threads per block), en vervolgens berekent men het aantal blokken zodat alle elementen verwerkt worden:

```
numBlocks = (n + threadsPerBlock - 1) / threadsPerBlock;
```

Hierbij is `n` het aantal elementen in de array.
Een te klein aantal threads zorgt ervoor dat de GPU onderbenut wordt en de berekening trager verloopt, terwijl een te groot aantal threads geen extra voordeel oplevert. De optimale configuratie ligt dus in het evenwicht tussen efficiënt geheugen- en kernelgebruik.

1.4.2 How does a thread know which element it should process?

Elke thread bepaalt zelf welk element ze moet verwerken aan de hand van haar unieke index. Deze index wordt berekend met de formule `idx = threadIdx.x + blockIdx.x * blockDim.x`.
Hierbij geeft `threadIdx.x` de positie van de thread binnen haar block aan, `blockIdx.x` de positie van het block binnen de grid, en `blockDim.x` het aantal threads per block. Door deze waarden te combineren, krijgt elke thread een unieke globale index binnen het volledige probleem. Op die manier weet elke thread exact welk element van de array ze moet lezen of schrijven, zonder overlap met andere threads.

1.4.3 What is the largest possible block size (max. threads per block)?

De grootste block size wordt bepaald door de hardware van de GPU. Voor de meeste moderne NVIDIA-GPU's ligt deze limiet op **1024 threads per block**. Deze waarde kan echter variëren afhankelijk van het specifieke GPU-model.

1.5 Conclusie

In dit lab werd aangetoond hoe een eenvoudige bewerking, zoals het omdraaien van een array, efficiënt kan worden uitgevoerd met behulp van CUDA. De oefening illustreert het belang van de juiste configuratie van blokken en threads, het correct beheren van geheugen en het vermijden van out-of-bounds-toegang. Deze inzichten vormen de basis voor complexere parallelle algoritmes die optimaal gebruik maken van de rekenkracht van de GPU.

2 Code

```cpp
#include <iostream>
#include <cuda_runtime.h>


__global__ void reverse_list(int *list, int *reverse, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < n){
        reverse[idx] = list[n - idx -1];
    }
}

int main() {
    const int arraySize = 10;
    int a[arraySize], r[arraySize];

    int *p_a, *p_r;

    for (int i = 0; i < arraySize; i++){
        a[i] = i;
    }


    cudaMalloc((void**)&p_a, sizeof(int)*arraySize);
    cudaMalloc((void**)&p_r, sizeof(int)*arraySize);
    cudaMemcpy(p_a, &a, arraySize * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(p_r, &r, arraySize * sizeof(int), cudaMemcpyHostToDevice);


    int numBlocks = 3;
    int threadsPerBlock = (arraySize + numBlocks - 1) / numBlocks;
    reverse_list<<<numBlocks, threadsPerBlock>>>(p_a, p_r, arraySize);
    reverse_list<<<numBlocks, threadsPerBlock>>>(p_a, p_r, arraySize);

    cudaMemcpy(&r, p_r, sizeof(int)*arraySize, cudaMemcpyDeviceToHost);

    std::cout << "Initial␣array:␣";
    for (int i = 0; i < arraySize; i++){
        std::cout << a[i];
    }
    std::cout << std::endl;


    std::cout << "Reverse␣array:␣";
    for (int i = 0; i < arraySize; i++){
        std::cout << r[i];
    }
    std::cout << std::endl;


    cudaFree(p_a);
    cudaFree(p_r);

    return 0;
}
```

Listing 1: Eenvoudige CUDA kernel